

Summer 2023

Subject: - Telecommunication Project

Report Topic: - Assignment II

Supervised By: - Dr.-Ing. Toralf Renkwitz

Team Members:

Lohith Reddy Parne (431031)

Srinivasa Prasad Tharun Kumar Chandrayudula Ramachandra (433303)

Hochschule Wismar

**University of Applied Sciences Technology, Business and Design,
Wismar, Germany**

**Faculty of Engineering Department of Electrical Engineering and
Computer Science**

INTRODUCTION

1:(a,b)

Fitting:

Python is a powerful tool for fitting data to any functional form. You are not restricted to the simple linear or polynomial functions that may fit in a spreadsheet program. For each parameter in a functional fit, you may also figure out the standard error.

Curve fitting:

Curve fitting is an optimization problem that finds a line that best fits a collection of observations. It is easiest to think about curve fitting in two dimensions, such as a graph. Consider that we have collected examples of data from the problem domain with inputs and outputs.

The independent variable or function's input is represented by the x-axis. The dependent variable or function's output is represented by the y-axis. Although we are unsure of the exact shape of the function that converts examples of inputs into outputs, we believe that we can estimate it using a standard form. [\(1\)](#)

Best fit:

A line that best describes the connection between the x-axis and y-axis coordinates of the data points presented as a scatter plot on a two-dimensional graph is known as the best fit line. [\(2\)](#)

Residuals:

A residual is the difference between an observed value and a predicted value in a regression model. It is calculated as:

Residual = Observed value – Predicted value. [\(3\)](#)

2a : Sinusoidal

Created on Tue Apr 25 23:17:57 2023

@author: parne

''''''

```
import random
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Define the minimum and maximum temperature values
min_temp = 5.0 # Celsius
max_temp = 25.0 # Celsius

# Generate a set of random temperature data
num_points = 50
temp_data = np.array([random.uniform(min_temp, max_temp) for i in range(num_points)])

# Define the sinusoidal function to fit
def sinusoid(x, A, W, c, d):
    return A * np.sin(W * x + c) + d

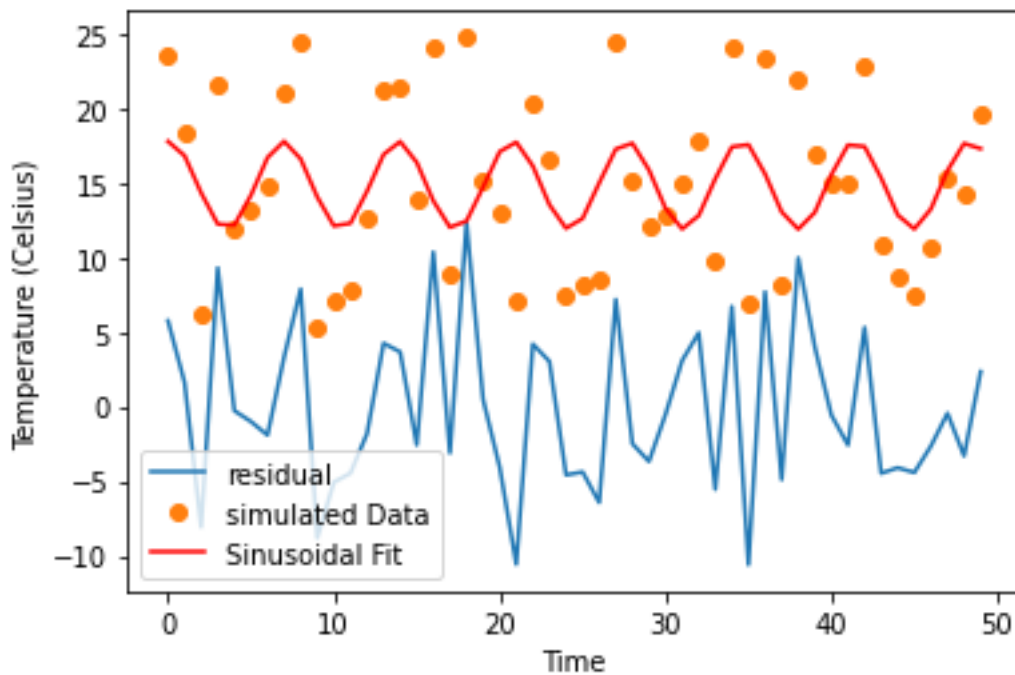
#noise
noise=np.random.randn(num_points)
#plt.plot(noise)

# Fit the sinusoidal function to the temperature data
x_data = np.arange(num_points)
popt, pcov = curve_fit(sinusoid, x_data, temp_data)

#residual
resud= temp_data -sinusoid(x_data, *popt)

# Plot the temperature data and the fitted sinusoidal function
plt.plot(noise,label= "noise")
plt.plot(resud, label = "residual")
plt.plot(x_data, temp_data, 'o', label='simulated Data')
plt.plot(x_data, sinusoid(x_data, *popt), 'r-', label='Sinusoidal Fit')
plt.xlabel("Time")
plt.ylabel("Temperature (Celsius)")
plt.legend()
plt.show()
#plt.savefig('sinusoidal.png')
```

graph:



Temperature measured in a garden shed for multiple days, considering a minimum temperature of 5 degrees C and a maximum temperature of 25 degrees C. 50 random temperatures are generated. The graph is plotted between temperature and number of days. Plotting the temperatures for multiple days and then fitting the data into a sinusoidal fit.

2b : Exponential

Created on Tue Apr 25 23:17:57 2023

@author: parne

''''''

```
import random
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt
```

```
# Define the time range and number of data points
t_start = 0.0 # Start time
t_end = 10.0 # End time
#num_points = 20 # Number of data points
```

Internal

```

# Create the time array
time = np.arange(t_start, t_end, 0.1)

# Define the initial speed and deceleration rate
initial_speed = 30.0 # km/h
deceleration_rate = 1 # m/s^2

# Calculate the corresponding speed values for each time point
speed = initial_speed * np.exp(-deceleration_rate * time)

# add noise to the velocity data
noise = np.random.normal(0, 10, size=len(time))
vel_data = speed + noise

# Define the exponential function to fit
def exp_func(x, a, b):
    return a * np.exp(-b * x)

# Fit the exponential function to the data
x_data = np.arange(num_points)
popt, pcov = curve_fit(exp_func, time, vel_data)

#residual
resud= vel_data - exp_func(time, *popt)

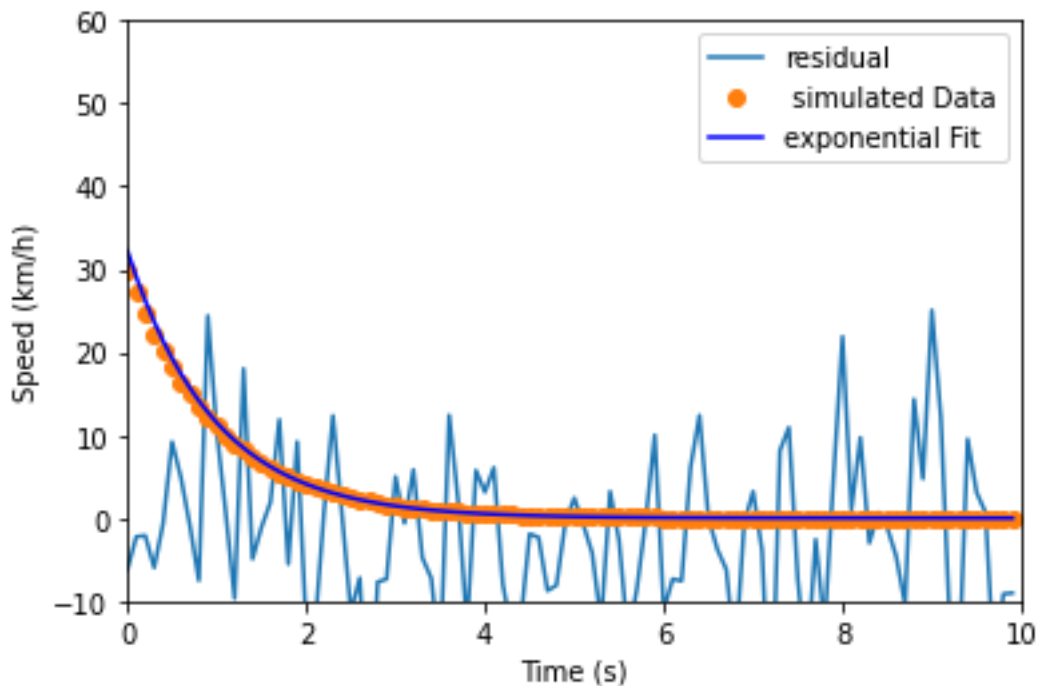
# Plot the data and the fitted curve

plt.plot(time,resud,"-", label ="residual")
plt.plot(time, speed, 'o', label=' simulated Data')
plt.plot(time, exp_func(time, *popt), 'b-', label='exponential Fit')
plt.xlabel('Time (s)')
plt.ylabel('Speed (km/h)')
plt.legend()

plt.xlim(0, 10)
plt.ylim(-10, 60)
plt.show()
#plt.savefig('exponential.png')

```

Graph:



Assuming a car with an initial speed of 30kmph and decelerating at a speed of 1kmph. Time taken for the car to stop is 10 seconds in the span of 20 points. Corresponding values of speed are calculated for each time point. Plotting the decelerating speeds with respect to time and fitting the data into exponential fit.

2c : Gaussian

Created on Tue Apr 25 23:17:57 2023

@author: parne

"""

```
import random
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Define the Gaussian function for fitting
def gauss_func(x, a, b, c):
    return a * np.exp(-(x - b)**2 / (2 * c**2))

# Generate the artificial data
values = [ 2, 3, 4, 5, 6, 7, 8, 9]
likelihoods = [ 1, 1, 1, 1, 2, 4, 2, 1]
occurrences = np.repeat(values, likelihoods)
```

```

noisy_occurrences = occurrences + np.random.normal(0, 0.5, size=len(occurrences))

# Perform curve fitting
data= np.arange(0.5, 10.5, 1)
hist, data = np.histogram(noisy_occurrences, bins=data)
data_centers = (data[:-1] + data[1:]) / 2
popt, pcov = curve_fit(gauss_func, data_centers, hist, p0=[len(noisy_occurrences)/10, 5, 1])

# Plot the residuals
residuals = hist - gauss_func(data_centers, *popt)

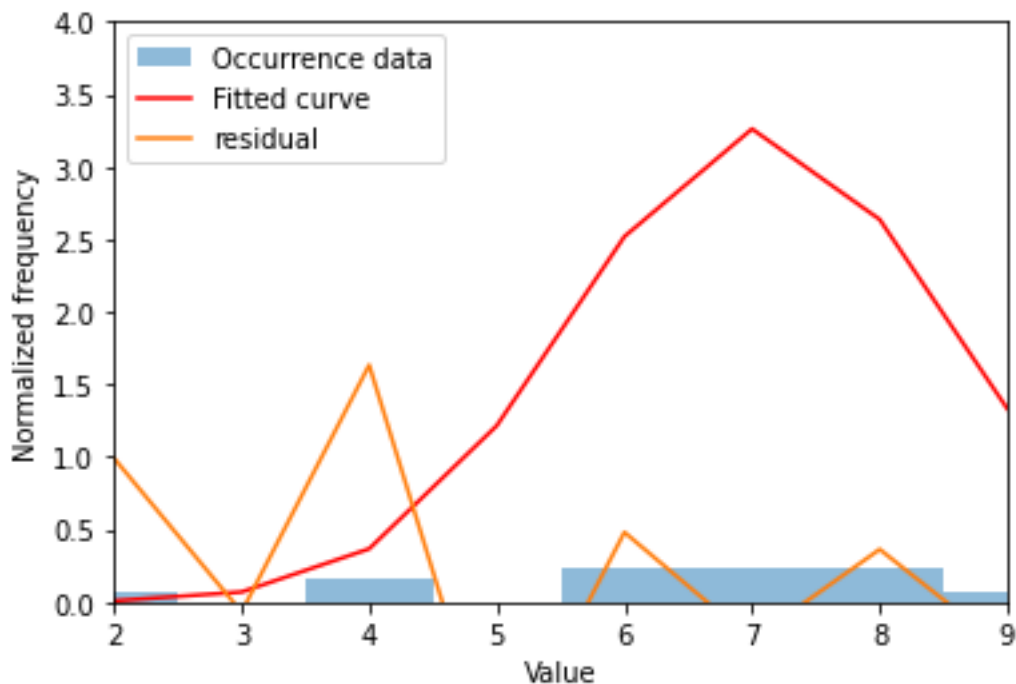
# Plot the data and fitted curve

plt.hist(noisy_occurrences, bins=data, density=True, alpha=0.5, label='Occurrence data')
plt.plot(data_centers, gauss_func(data_centers, *popt), 'r-', label='Fitted curve')
plt.plot(data_centers, residuals, '-o', label='residual')
plt.xlabel('Value')
plt.ylabel('Normalized frequency')
plt.legend()
plt.xlim(2,9)
plt.ylim(0,4)

#plt.savefig('gaussian.png')

```

Graph:



When a super dice is rolled, the outcomes possible are between 1 and 10 i.e., 2,3,4,5,6,7,8 and 9. But 6 and 8 have 2x and 7 has 4x the likelihood of the other. Probabilities of each outcome is calculated.

$$P(2)=P(3)=P(4)=P(5)=P(9)=x$$

$$P(6)=P(8)=2x, P(7)=4x$$

As we know, the sum of probabilities of each outcome is equals to 1.

$$\sum_{i=2}^9 P(i) = 1$$

$$x+x+x+x+2x+4x+2x+x=1$$

$$13x=1$$

$$x=1/13$$

Mean and variance are also calculated. An array of possible outcomes is created. Gaussian distribution is plotted and a graph between Probability Mean Function and Gaussian distribution is plotted.

2d: Linear

Created on Tue Apr 25 23:17:57 2023

@author: parne
''''''

```
import random
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Constants
g = 9.81
t = np.arange(0,10,0.1) # Time in seconds

# Generate altitude data with noise
slope = -9.8 / 2 # Acceleration due to gravity
intercept = 50 # Starting altitude
data = intercept + slope * t + np.random.normal(0, 5, size=len(t))

# Calculate height
h = 0.5 * g * t**2

# Define linear function
def linear_func(x, a, b):
    return a * x + b
```

```

# Fit linear function to noisy data
popt, pcov = curve_fit(linear_func, t, data)

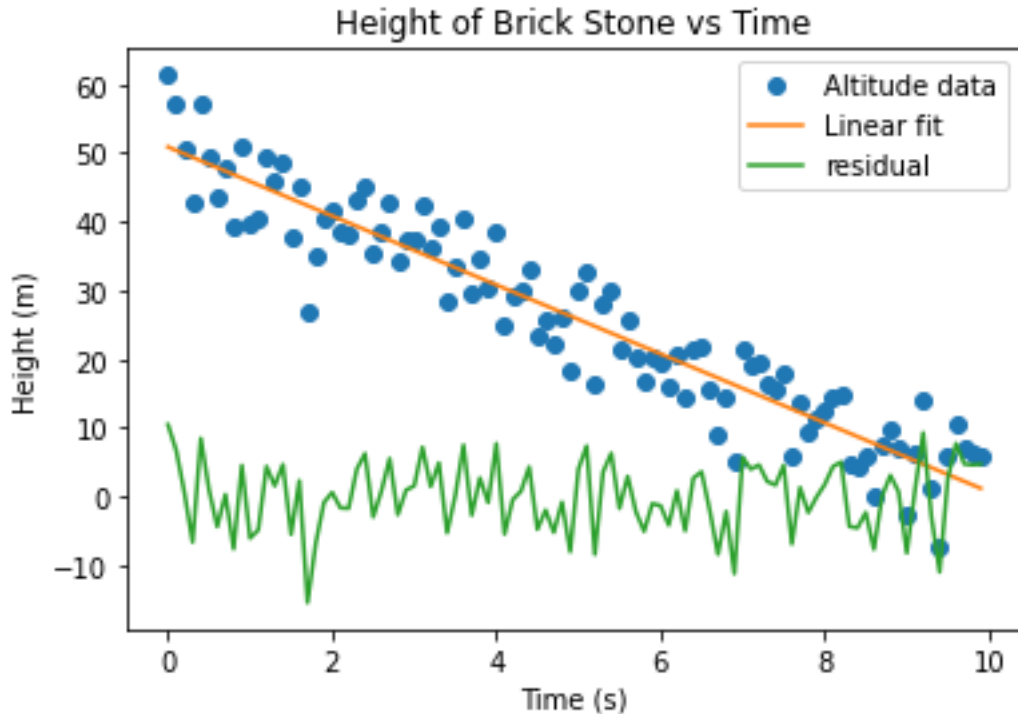
#residuals
residuals = data - linear_func(t, *popt)

# Generate linear line using fitted parameters
line = linear_func(t, *popt)

# Plot data and linear function
plt.plot(time, data, 'o', label='Altitude data')
plt.plot(t, line, label='Linear fit')
plt.plot(t, residuals, '-', label='residual')
plt.xlabel('Time (s)')
plt.ylabel('Height (m)')
plt.title('Height of Brick Stone vs Time')
plt.legend()
plt.show()
#plt.savefig('linear.png')

```

Graph:



Assuming a brick stone falling from the roof of a height of 122.5m. The brick does not collide and just a light air friction occurs. Distance travelled by the brick in 't' seconds is $s = \frac{1}{2}gt^2$ as initial velocity is 0 m/s².

Distance travelled by the brick after 0s, 1s, 2s, 3s, 4s and 5s is 0m, 4.9m, 19.6m, 44.1m, 78.4m and 122.5m, respectively. The graph is plotted between altitude and time. A linear graph is obtained which is the best fit.

References:

1. <https://machinelearningmastery.com/curve-fitting-with-python/>
2. <https://pythonguides.com/matplotlib-best-fit-line/#:~:text=The%20best%20fit%20line%20or,points%20from%20the%20purposed%20line.&text=Here%2C%20x%20and%20y%20are,and%20C%20is%20the%20constant.>
3. <https://www.statology.org/standardized-residuals-python/#:~:text=A%20residual%20is%20the%20difference,Residual%20%3D%20Observed%20value%20-%20Predicted%20value>
4. <https://studip.hs-wismar.de/dispatch.php/course/overview?cid=b3bb2f3c71d55a6ec368d250f067a73f--> "Dr.-Ing. Toralf Renkwitz" (studip code)